

Concurrency

– это система организации параллельной работы(с т.з. ПК)

Fan-In & Fan-Out каналы

Аналогия для понимания.

Fan-Out – это когда вы берете одну деталь и отдаете ее 10 станкам одновременно

Fan-In – это когда вы берете готовые изделия с 10 станков и кладете их в одну коробку

Fan-Out (Разветвление)

Суть: У вас есть один входной канал, из которого читают данные множество горутин (воркеров).

Это делается для распараллеливания тяжелых задач (CPU-bound, I/O-bound)

Правило: Каждая горутина читает из одного и того же канала. Канал отдает каждое значение только одному читателю (это гарантировано мьютексом внутри канала)

```
package main

import (
    "fmt"
    "sync"
    "time"
)

// Тяжелая работа, которую хотим распараллелить
func worker(id int, jobs <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs { // Читаем из общего канала, пока он не закроется
        fmt.Printf("Воркер %d обрабатывает задачу %d\n", id, job)
        time.Sleep(time.Second) // Симулируем нагрузку
    }
}

func main() {
    const numWorkers = 5
    jobs := make(chan int, 10) // Буферизированный канал для задач

    var wg sync.WaitGroup

    // --- FAN-OUT: Запускаем 5 горутин, которые читают из ОДНОГО канала ---
    for i := 1; i <= numWorkers; i++ {
        wg.Add(1)
        go worker(i, jobs, &wg)
    }

    // Отправляем 10 задач в канал
    for i := 1; i <= 10; i++ {
        jobs <- i
    }
    close(jobs) // Закрываем канал, чтобы воркеры вышли из цикла range

    wg.Wait()
    fmt.Println("Все задачи выполнены")
}
```

При этом порядок в котором будут обработаны задачи недетерминирован, Race Condition.

Причины:

1. Jitter (дрожание) запуска горутин. Горутины стартуют не атомарно
2. Fairness (честность) каналов. В Go при выборе, кому отдать данные из канала, если ждут несколько горутин, используется псевдослучайный алгоритм, сделано для избежания голоданий.
3. Работа планировщика ОС. Go рантайм просит кванты времени на потоки М. ОС может дать квант сначала потоку М1, потом М2, а может наоборот. Это чистая лотерея

Fan-In (Схождение)

Суть:

У нас есть много каналов, из которых приходят результаты. Мы хотим собрать их все в один результирующий канал. Чтобы читать результаты по мере их поступления (как только любой воркер закончил)

Чтобы слушать каналы одновременно, будем использовать `select` в бесконечном цикле.

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

// Симуляция долгого запроса к БД или API
func fetchData(sourceID int) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        delay := time.Duration(rand.Intn(500)) * time.Millisecond
        time.Sleep(delay)
        out <- fmt.Sprintf("Результат от источника %d (ждял %v)", sourceID, delay)
    }()
    return out
}

// --- FAN-IN: Слияние множества каналов в один ---
func merge(channels ...<-chan string) <-chan string {
    out := make(chan string)

    // Запускаем горутину-сборщик
    go func() {
        var wg sync.WaitGroup

        // Для каждого входного канала запускаем свою горутину-прокладку
        // _ вместо переменной для индекса
        for _, ch := range channels {
            wg.Add(1)
            go func(c <-chan string) {
                defer wg.Done()
                for val := range c { // Читаем из дочернего канала
                    out <- val // Перекладываем значение в общий канал
                }
            }(ch)
        }
    }()
}
```

```

    }

    // Ждем, пока все дочерние каналы закроются, и закрываем общий
    wg.Wait()
    close(out)
}()

return out
}

func main() {
    // Запускаем 3 источника данных (Fan-Out неявно, но важен Fan-In)
    ch1 := fetchData(1)
    ch2 := fetchData(2)
    ch3 := fetchData(3)

    // Сливаем их в один канал
    merged := merge(ch1, ch2, ch3)

    // Читаем результаты по мере их готовности (кто первый прибежал)
    for result := range merged {
        fmt.Println(result)
    }
}

```

Координация: sync

Каналы – это способ обмена данными. Но иногда нужно просто скоординировать работу.

Data Race

```

package main

import "fmt"

var counter int

func increment() {
    // неатомарная операция (READ+WRITE)
    counter++
}

func main() {
    for i := 0; i < 1000; i++ {
        go increment()
    }

    fmt.Println(counter)
}

```

Мы увидим почти случайное число, а не желаемую 1000 Причина:

Операция counter++ состоит из 3 шагов:

1. Чтение текущего значения counter из памяти в регистр CPU
2. Прибавление 1
3. Запись нового значения обратно в память

Если 2 горутины исполнят шаг №1 одновременно, то обе увидят одинаковое и сделают одинаковое. В итоге не смотря на 2 горутин, число будет увеличено на 1. Таким образом и формируется “случайное” число.

Решение: операцию нужно сделать неделимой(вернее, атомарной).

sync.WaitGroup

```
type WaitGroup struct {  
    // noCopy - маркер, запрещающий копирование  
    noCopy noCopy  
  
    // state1 - основное состояние (счетчик 32b + семафор 32b)  
    state1 uint64  
  
    // state2 - дополнительное состояние (для 32-битных архитектур)  
    state2 uint32  
}
```

Этот механизм позволяет ждать группу горутин

```
package main  
  
import (  
    "fmt"  
    "sync"  
)  
  
func worker(id int, wg *sync.WaitGroup) {  
    defer wg.Done()  
    fmt.Printf("Worker %d started\n", id)  
}  
  
func main() {  
    var wg sync.WaitGroup  
  
    for i := 1; i <= 3; i++ {  
        wg.Add(1)  
        go worker(i, &wg)  
    }  
  
    wg.Wait()  
    fmt.Println("Все воркеры завершились")  
}
```

Важно: WaitGroup передается по указателю (&wg), иначе будет копия, и Done() не сработает

Важный пример на понимание:

```
package main  
  
import (  
    "fmt"  
    "sync"  
    "time"  
)  
  
func main() {  
    var wg sync.WaitGroup  
    wg.Add(1)  
  
    go func() {  
        fmt.Println("1. Начало работы")  
  
        wg.Done() // Счетчик стал 0  
    }()  
}
```

```
    fmt.Println("2. Done() вызван, продолжаем работу")

    // Какая-то тяжелая работа
    time.Sleep(2 * time.Second)
    fmt.Println("3. Завершение работы горутины")
}()

wg.Wait() // Ждет, пока счетчик станет 0
fmt.Println("4. main() продолжает работу")
}
```

У меня выводит так:

1. Начало работы
2. Done() вызван, продолжаем работу
4. main() продолжает работу

Дело в том, что `wg.Wait()` разблокируется как только счетчик станет равен 0. Вот счетчик стал равен 0 и за 2 секунды сна, горутина `main()` уже успела завершиться, в том числе поэтому очень важно использовать `defer wg.Done()`

Также при неправильном использовании `wg.Done()` может быть Data-race

Mutex / RWMutex

Правила:

1. Не копируйте мьютекс, он хранит внутреннее состояние (флаг блокировки, очередь ожидания). Копия нарушит логику. Передача только по указателю
2. Всегда разблокируйте. Используйте defer для гарантии разблокировки, особенно если в секции могут быть паника или ранний return.

но иногда лучше разблокировать вручную, без defer (минус накладные расходы)
3. Не блокируйте мьютекс повторно. Одна и та же горутинa не может вызвать Lock() дважды подряд без Unlock() – это приводит к deadlock (взаимная блокировка, если есть другие горутинy), либо к Fatal error all goroutines are asleep

sync.Mutex (Взаимное исключение)

```
// Упрощенная структура sync.Mutex
type Mutex struct {
    state int32 // Просто состояние (заблокирован/свободен)
    sema  uint32 // Семафор для ожидания
}

// Мьютекс НЕ знает, кто его захватил!
// Поэтому при повторном Lock он просто видит:
// "state != 0" -> блокируемся
```

– это самый простой мьютекс. Он гарантирует, что в критическую секцию одновременно может войти только одна горутинa

Методы:

- Lock() – лочит мьютекс. Если он уже залочен, то горутинa блокируется (спит), пока мьютекс не освободят
- Unlock() – анлочит мьютекс. Если вызвать на незаблокированном мьютексе – будет паника.

```
package main

import (
    "fmt"
    "sync"
)

var (
    counter int
    mu      sync.Mutex
)

func safeIncrement(wg *sync.WaitGroup) {
    defer wg.Done()

    mu.Lock()
    counter++
    mu.Unlock()
}

func main() {
    var wg sync.WaitGroup
```



```

    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go safeIncrement(&wg)
    }
    wg.Wait()
    fmt.Println(counter)
}

```

- Mutex не рекурсивный. В Go sync.Mutex не запоминает, какая горютина его захватила, и не считает количество захватов.

Вот так можно в Java, но не в Go:

```

// Java - рекурсивный мьютекс
public class Example {
    private final Object lock = new Object();

    public void outer() {
        synchronized(lock) {           // 1-й захват
            System.out.println("outer");
            inner();                     // Можно вызывать!
        }
    }

    public void inner() {
        synchronized(lock) {           // 2-й захват - ЭТО РАБОТАЕТ!
            System.out.println("inner");
        }
    }
}

```

sync.RWMutex (Читатель-Писатель)

RWMutex – это улучшенная версия. Он разделяет доступ на чтение и запись

```

type RWMutex struct {
    w          Mutex    // Мьютекс для писателей
    writerSem   uint32   // Семафор для писателей
    readerSem   uint32   // Семафор для читателей
    readerCount int32    // Количество активных читателей
    readerWait  int32    // Сколько читателей ждут писателя
}

```

Правила:

Читатели (Rlock/RUnlock):

- Могут входить одновременно сколько угодно, если нет активного писателя
- Блокируется, только если писатель уже захватил Lock()

Писатель (Lock/Unlock):

- Может войти только тогда, когда нет ни одного читателя и нет другого писателя
- Пока писатель внутри, новые читатели блокируются (ждут)

Методы:

- RLock() / RUnlock() – для чтения
- Lock() / Unlock() – для записи

Кэш с конкурентным доступом

```

package main

import (
    "fmt"
    "sync"
    "time"
)

type Cache struct {
    data map[string]string
    mu    sync.RWMutex
}

func NewCache() *Cache {
    return &Cache{data: make(map[string]string)}
}

// Чтение: много горутин могут читать одновременно
func (c *Cache) Get(key string) string {
    c.mu.RLock() // Блокировка на чтение
    defer c.mu.RUnlock()
    return c.data[key]
}

// Запись: эксклюзивный доступ
func (c *Cache) Set(key, value string) {
    c.mu.Lock() // Блокировка на запись
    defer c.mu.Unlock()
    c.data[key] = value
}

func main() {
    cache := NewCache()
    cache.Set("name", "Go")

    // Запускаем 10 читателей
    var wg sync.WaitGroup
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            fmt.Println(cache.Get("name"))
        }()
    }
    wg.Wait()
}

```

Особенности RWMutex

Голодание писателя (Writer Starvation)

– это ситуация, когда писатель не может захватить блокировку на запись, потому что постоянно приходят новые читатели

Согласно правилам, читатели могут входить, если нет **АКТИВНОГО** писателя

ВАЖНО: Писатель становится АКТИВНЫМ только после того, как успешно захватил Lock()

То есть, пока писатель ждет выхода старых читателей, могут зайти новые читатели, что и создает проблему

Как решать?

1. Шардирование: разбить данные на несколько независимых частей

```
type ShardedCache struct {
    shards [16]struct {
        mu sync.RWMutex
        data map[string]string
    }
}

func (c *ShardedCache) getShard(key string) int {
    hash := 0
    for _, ch := range key {
        hash += int(ch)
    }
    return hash % 16
}

func (c *ShardedCache) Get(key string) string {
    shardIdx := c.getShard(key)
    shard := &c.shards[shardIdx]

    shard.mu.RLock()
    defer shard.mu.RUnlock()
    return shard.data[key]
}

func (c *ShardedCache) Set(key, value string) {
    shardIdx := c.getShard(key)
    shard := &c.shards[shardIdx]

    shard.mu.Lock()
    defer shard.mu.Unlock()
    shard.data[key] = value
}
```

Преимущество: Писатель блокирует только один шард, а не весь кэш.

2. Использование каналов с очередью

```
type Operation struct {
    IsWrite bool
    Key     string
    Value   string
    Result  chan string
}

type SafeMap struct {
    data map[string]string
    ops  chan Operation
    stop chan struct{}
}

func NewSafeMap() *SafeMap {
```

```

    sm := &SafeMap{
        data: make(map[string]string),
        ops:  make(chan Operation, 1000),
        stop: make(chan struct{}),
    }
    go sm.process()
    return sm
}

func (sm *SafeMap) process() {
    for {
        select {
        case op := <-sm.ops:
            if op.IsWrite {
                // Запись имеет приоритет
                sm.data[op.Key] = op.Value
            } else {
                // Чтение
                op.Result <- sm.data[op.Key]
            }
        case <-sm.stop:
            return
        }
    }
}

func (sm *SafeMap) Get(key string) string {
    ch := make(chan string, 1)
    sm.ops <- Operation{IsWrite: false, Key: key, Result: ch}
    return <-ch
}

func (sm *SafeMap) Set(key, value string) {
    sm.ops <- Operation{IsWrite: true, Key: key, Value: value}
}

```

Нерекурсивность, а повторяемость Rlock()

Рекурсивный мьютекс (или reentrant mutex) – это мьютекс, который запоминает, какой поток его захватил, и позволяет этому же потоку захватывать его сколько угодно раз, пока она его держит

Такой мьютекс:

- имеет счетчик захватов (recursion count)
- имеет владельца (owner) – поток
- если владелец пытается захватить мьютекс повторно, то просто увеличивается счетчик
- если другой владелец пытается захватить мьютекс, то происходит блокировка

А в Go владелец не отслеживается, поэтому несмотря на то, что можно повторно вызвать RLock() и RUnlock(), такой мьютекс не считается рекурсивным. (работает он через счетчик readerCount, но не путать с счетчиком рекурсии)

```
package main
```

```
import (
    "fmt"
    "sync"

```

```

)

var rwmu sync.RWMutex

func read1() {
    rwmu.RLock()
    defer rwmu.RUnlock()

    read2() // Можно! RLock рекурсивен
}

func read2() {
    rwmu.RLock()
    defer rwmu.RUnlock()

    fmt.Println("Вложенное чтение")
}

func main() {
    read1() // Работает!
}

```

Когда что использовать?

Используйте RWMutex, если:

- Чтений значительно больше, чем записей (пропорция 10:1 или выше).
- Чтение выполняется долго (например, сериализация JSON, чтение из БД).
- Вы хотите повысить производительность за счет параллельных чтений.

Используйте обычный Mutex, если:

- Записи и чтения примерно равны по частоте.
- Критическая секция очень короткая (меньше, чем накладные расходы на RWMutex).
- RWMutex сложнее в использовании и может привести к зависаниям (см. ниже).

sync.Once

– это примитив синхронизации, который гарантирует разовое выполнение (мастхев для безопасной ленивой инициализации глоб переменных, подключений к базе данных или паттерна Singleton)

Устройство:

```
type Once struct {
    done uint32    // атомарный флаг: 0 – не выполнено, 1 – выполнено
    m     Mutex     // мьютекс для синхронизации
}
```

Сценарий 1: Последовательность вызовов (без горутинов)

```
func main() {
    var once sync.Once

    once.Do(func() { fmt.Println("Первый") })
    once.Do(func() { fmt.Println("Второй") })
    once.Do(func() { fmt.Println("Третий") })
}
```

Вывод:

Первый

Сценарий 2: Смешанные вызовы

```
func main() {
    var once sync.Once
    var wg sync.WaitGroup

    once.Do(func() { fmt.Println("Обычный вызов") })

    for i := 0; i < 5; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            once.Do(func() {
                fmt.Printf("Горутина %d\n", id)
            })
        }(i)
    }

    wg.Wait()
}
```

Вывод:

Обычный вызов

Сценарий 3: Паника внутри Once

```
func main() {
    var once sync.Once

    defer func() {
        if r := recover(); r != nil {
            fmt.Println("Поймали панику:", r)
        }
    }
}
```

```

    }()

    // 1
    once.Do(func() {
        fmt.Println("Выполняем...")
        panic("ошибка!")
    })

    // Этот вызов выполнится!
    once.Do(func() {
        fmt.Println("Повторная попытка")
    })
}

```

Вывод:

```

Выполняем...
Поймали панику: ошибка!
Повторная попытка

```

Если колбэк паникует, флаг done НЕ устанавливается в 1. Поэтому при следующем вызове Do() функция выполнится снова. Это сделано специально, чтобы дать шанс восстановиться.

Сценарий 4. Антипаттерн. Бесконечная блокировка

```

func main() {
    var once sync.Once

    once.Do(func() {
        fmt.Println("Начинаем...")
        // Опасный рекурсивный вызов!
        once.Do(func() {
            fmt.Println("Вложенный вызов")
        })
    })
}

```

Получим Deadlock. Почему?

1. Первый вызов захватил мьютекс
2. Внутри колбэка происходит второй вызов Do()
3. Второй вызов видит done == 0, пытается захватить мьютекс
4. Мьютекс уже захвачен первым вызовом -> взаимная блокировка

Сценарий 5. Мутабельный синглтон

```

var config *Config
var once sync.Once

func GetConfig() *Config {
    once.Do(func() {
        config = &Config{Port: 8080}
    })
    return config
}

// Где-то в коде:
func handler1() {
    cfg := GetConfig()
}

```

```

    cfg.Port = 9090 // Меняем глобальное состояние!
}

func handler2() {
    cfg := GetConfig()
    fmt.Println(cfg.Port) // 9090 — поведение неожиданное
}

```

Чтобы избежать проблем, можно вернуть копию (*config)

Сценарий 6. Подключение к БД

```

var (
    db *sql.DB
    once sync.Once
)

func GetDB() *sql.DB {
    once.Do(func() {
        // Подключение к БД — это ресурс, который должен быть ОДНИМ
        // Это НЕ антипаттерн!
        db, _ = sql.Open("postgres", os.Getenv("DATABASE_URL"))
        db.SetMaxOpenConns(10)
    })
    return db
}

// Почему это безопасно?
// 1. *sql.DB потокобезопасен
// 2. Мы НЕ меняем подключение (не переопределяем)
// 3. Это разделяемый ресурс, которым пользуются все

```

Также полезно использовать этот примитив синхронизации для инициализаций логгера, метрик и тп.

sync.Pool

sync.Pool – это временное хранилище объектов, чтобы не создавать новые и снизить нагрузку на GC.

```
type Pool struct {
    // публичное поле -- фабрика
    New func() interface{}

    // приватные поля
    // локальный пул -- указатель на массив poolLocal
    // каждый P (процессор) имеет собственный пул
    local    unsafe.Pointer // указатель на []poolLocal
    localSize uintptr        // размер локального массива

    // victim -- жертвенный пул для смягчения GC
    // объекты, которые пережили один цикл GC
    victim    unsafe.Pointer
    victimSize uintptr

    // для защиты в редких случаях
    mu sync.Mutex
}
```

Что такое poolLocal?

Каждый процессор (P) в Go runtime имеет свой локальный пул объектов:

```
type poolLocal struct {
    poolLocalInternal
    // выравнивание для предотвращения
    // ложного совместного доступа (false sharing)
    pad [128 - unsafe.Sizeof(poolLocalInternal{})%128]byte
}

type poolLocalInternal struct {
    // быстрый доступ к одному объекту (без блокировок)
    private interface{}
    // очередь объектов для этого P (с блокировками)
    shared poolChain
}

type poolChain struct {
    head *poolChainElt // начало очереди (только push/pop с одной стороны)
    tail *poolChainElt // конец очереди (для кражи)
}

type poolChainElt struct {
    poolDequeue // кольцевой буфер фиксированного размера
    next, prev *poolChainElt
}
```

Пример для ознакомления:

```
package main

import (
    "fmt"
    "sync"
```

```

)

func main() {
    pool := &sync.Pool{
        New: func() interface{} {
            fmt.Println("Создаём новый объект")
            return make([]byte, 1024)
        },
    }

    // Получаем объект из пула
    buf := pool.Get().([]byte)
    fmt.Println("Получили:", len(buf))

    // Используем
    buf = append(buf, 42)

    // Возвращаем в пул
    pool.Put(buf)

    // Берём снова – получаем тот же объект
    buf2 := pool.Get().([]byte)
    fmt.Println("Снова получили:", len(buf2))
}

```

Вывод:

Создаём новый объект

Получили: 1024

Снова получили: 1025

Структура и методы на практике

Вернемся к насущному

```

type Pool struct {
    New func() interface{} // фабрика (не метод, но ключевая часть)
}

func (p *Pool) Get() interface{} // взять объект
func (p *Pool) Put(x interface{}) // вернуть объект

```

Поле New – фабрика объектов

```

var pool = sync.Pool{
    New: func() interface{} {
        return &MyStruct{}
    },
}

```

Что и зачем:

- Функция – фабрика нового объекта
- Вызывается автоматически в Get(), если пул пуст
- Не обязательна, но очень рекомендуется

Правила:

- Если New == nil, то Get() вернёт nil, когда пул пуст.
- Должна возвращать новый объект (не из пула).

- Должна быть потокобезопасной (обычно это просто конструктор).

Метод `Get()` – взять объект

```
func (p *Pool) Get() interface{}
```

Что делает:

- Возвращает объект из пула (если есть).
- Если пул пуст – создаёт новый через `New()`.
- Если `New == nil` и пул пуст – возвращает `nil`.

Рекомендации:

1. Type assertion

```
// Неправильно: работаем с interface{}
obj := pool.Get()
obj.WriteString("hello") // ОШИБКА! interface{} не имеет метода WriteString

// Правильно: приводим к нужному типу
buf := pool.Get().(*bytes.Buffer)
buf.WriteString("hello")
```

2. Объект нужно сбросить.

```
// Неправильно: предполагаем, что объект чистый
buf := pool.Get().(*bytes.Buffer)
buf.WriteString("data") // может дописать к старым данным!

// Правильно: сбрасываем состояние перед использованием
buf := pool.Get().(*bytes.Buffer)
buf.Reset() // критически важно!
buf.WriteString("data")
```

Но сначала убедись что `Reset()` существует, например, если работаем со своими структурами или типами без такого метода.

3. Не хранить ссылку на объект после использования
4. `Get()` может вернуть объект, созданный другим потоком. Это нормально. Все равно надо сбросить его состояние

Метод `Put()` – вернуть объект

```
func (p *Pool) Put(x interface{})
```

Он кладет объект обратно в пул для будущего переиспользования

Правила+Рекомендации+Возможности:

1. Кладем то, что взяли

```
// Неправильно: кладем чужой объект
buf := &bytes.Buffer{} // создали сами
pool.Put(buf) // можно, но НЕ РЕКОМЕНДУЕТСЯ

// Правильно: кладем только взятое из пула
buf := pool.Get().(*bytes.Buffer)
// ... работа
pool.Put(buf) // возвращаем то же, что взяли
```

2. Объект должен быть “чистым”:

```
// Неправильно: кладем "грязный" объект
buf := pool.Get().(*bytes.Buffer)
```

```

buf.WriteString("data")
pool.Put(buf) // следующий получит буфер с "data"

// Правильно: сбрасываем перед возвратом (опционально)
buf := pool.Get().(*bytes.Buffer)
buf.WriteString("data")
buf.Reset() // чистим!
pool.Put(buf)

```

3. Можно не возвращать объект, но тогда он уйдет в GC
4. Можно передать nil, но это бессмысленно (он попросту не будет храниться)

Пример хорошей практики:

```

package main

import (
    "sync"
    "bytes"
)

var pool = sync.Pool{
    New: func() interface{} {
        return &bytes.Buffer{}
    },
}

func processData(data string) {
    // 1. Забираем объект
    buf := pool.Get().(*bytes.Buffer)

    // 2. Сбрасываем состояние (КРИТИЧНО!)
    buf.Reset()

    // 3. Используем
    buf.WriteString(data)
    result := buf.String()

    // 4. Возвращаем в пул (через defer для безопасности)
    defer pool.Put(buf)

    // Используем result
    _ = result
}

```

Идиомы и лучшие практики

1. Всегда объявляйте New

```

var pool = sync.Pool{
    New: func() interface{} {
        return make([]byte, 0, 1024)
    },
}

```

2. Используйте defer для Put()

```

buf := pool.Get().(*bytes.Buffer)
defer pool.Put(buf)

```

3. Сбрасывайте объект сразу после Get()

```
buf := pool.Get().(*MyStruct)
buf.Reset() // или очистка полей
```

4. Не храните ссылки на объекты после Put()

```
buf := pool.Get().(*bytes.Buffer)
defer pool.Put(buf)
// ... работа
// после Put не используйте buf!
```

5. Проверяйте тип при Get()

```
// Небезопасно
obj := pool.Get()
buf := obj.(*bytes.Buffer) // может упасть!

// Безопасно
obj := pool.Get()
buf, ok := obj.(*bytes.Buffer)
if !ok {
    // обработайте ошибку
}
```

Особый случай

С sync.Pool можно работать без New:

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var pool sync.Pool

    // 1. Кладём объект
    pool.Put("first message")

    // 2. Забираем его
    msg1 := pool.Get()
    fmt.Println(msg1) // "first message"

    // 3. Пул снова пуст -> Get() вернёт nil
    msg2 := pool.Get()
    fmt.Println(msg2) // nil (так как New == nil)
}
```

Но это не значит что так надо делать

Concurrency Patterns

Fan-In / Fan-Out

Аналогия для понимания.

Fan-Out – это когда вы берете одну деталь и отдаете ее 10 станкам одновременно

Fan-In – это когда вы берете готовые изделия с 10 станков и кладете их в одну коробку

Fan-Out (Разветвление)

Суть: У вас есть один входной канал, из которого читают данные множество горутин (воркеров).

Это делается для распараллеливания тяжелых задач (CPU-bound, I/O-bound)

Правило: Каждая горутина читает из одного и того же канала. Канал отдает каждое значение только одному читателю (это гарантировано мьютексом внутри канала)

```
package main

import (
    "fmt"
    "sync"
    "time"
)

// Тяжелая работа, которую хотим распараллелить
func worker(id int, jobs <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs { // Читаем из общего канала, пока он не закроется
        fmt.Printf("Воркер %d обрабатывает задачу %d\n", id, job)
        time.Sleep(time.Second) // Симулируем нагрузку
    }
}

func main() {
    const numWorkers = 5
    jobs := make(chan int, 10) // Буферизированный канал для задач

    var wg sync.WaitGroup

    // --- FAN-OUT: Запускаем 5 горутин, которые читают из ОДНОГО канала ---
    for i := 1; i <= numWorkers; i++ {
        wg.Add(1)
        go worker(i, jobs, &wg)
    }

    // Отправляем 10 задач в канал
    for i := 1; i <= 10; i++ {
        jobs <- i
    }
    close(jobs) // Закрываем канал, чтобы воркеры вышли из цикла range

    wg.Wait()
    fmt.Println("Все задачи выполнены")
}
```

При этом порядок в котором будут обработаны задачи недетерминирован, Race Condition.

Причины:

1. Jitter (дрожание) запуска горутин. Горутины стартуют не атомарно
2. Fairness (честность) каналов. В Go при выборе, кому отдать данные из канала, если ждут несколько горутин, используется псевдослучайный алгоритм, сделано для избежания голоданий.
3. Работа планировщика ОС. Go рантайм просит кванты времени на потоки М. ОС может дать квант сначала потоку М1, потом М2, а может наоборот. Это чистая лотерея

Fan-In (Схождение)

Суть:

У нас есть много каналов, из которых приходят результаты. Мы хотим собрать их все в один результирующий канал. Чтобы читать результаты по мере их поступления (как только любой воркер закончил)

Чтобы слушать каналы одновременно, будем использовать `select` в бесконечном цикле.

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

// Симуляция долгого запроса к БД или API
func fetchData(sourceID int) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        delay := time.Duration(rand.Intn(500)) * time.Millisecond
        time.Sleep(delay)
        out <- fmt.Sprintf("Результат от источника %d (ждял %v)", sourceID, delay)
    }()
    return out
}

// --- FAN-IN: Слияние множества каналов в один ---
func merge(channels ...<-chan string) <-chan string {
    out := make(chan string)

    // Запускаем горутины-сборщик
    go func() {
        var wg sync.WaitGroup

        // Для каждого входного канала запускаем свою горутины-прокладку
        // _ вместо переменной для индекса
        for _, ch := range channels {
            wg.Add(1)
            go func(c <-chan string) {
                defer wg.Done()
                for val := range c { // Читаем из дочернего канала
                    out <- val // Перекладываем значение в общий канал
                }
            }(ch)
        }
    }()
}
```

```

    }

    // Ждем, пока все дочерние каналы закроются, и закрываем общий
    wg.Wait()
    close(out)
}()

return out
}

func main() {
    // Запускаем 3 источника данных (Fan-Out неявно, но важен Fan-In)
    ch1 := fetchData(1)
    ch2 := fetchData(2)
    ch3 := fetchData(3)

    // Сливаем их в один канал
    merged := merge(ch1, ch2, ch3)

    // Читаем результаты по мере их готовности (кто первый прибежал)
    for result := range merged {
        fmt.Println(result)
    }
}

```

Worker Pool

Суть в создании фиксированного количества горутин (воркеров), которые читают задачи из одного общего канала

1. Классическая архитектура (Каналы + sync.WaitGroup)

```

package main

import (
    "fmt"
    "sync"
)

func worker(id int, jobs <-chan int, results chan<- int, wg *sync.WaitGroup) {
    defer wg.Done() // Сообщаем, что воркер завершился
    for job := range jobs { // Цикл завершится, когда канал закроют
        fmt.Printf("Worker %d processing job %d\n", id, job)
        results <- job * 2 // Имитация работы
    }
}

func main() {
    const numWorkers = 3
    jobs := make(chan int, 10)
    results := make(chan int, 10)

    var wg sync.WaitGroup
    wg.Add(numWorkers)

    // Запускаем пул
    for i := 0; i < numWorkers; i++ {
        go worker(i, jobs, results, &wg)
    }
}

```



```

}

// Отправляем задачи
for i := 0; i < 5; i++ {
    jobs <- i
}
close(jobs) // Без закрытия воркеры зависнут в deadlock

// Ждем завершения ВСЕХ воркеров
wg.Wait()
close(results) // Теперь можно закрыть канал результатов

// Читаем результаты
for res := range results {
    fmt.Println("Result:", res)
}
}

```

Важно понимать, Worker Pool очень похож на Fan-Out/In, это не спроста

Worker-pool это бизнес-паттерн, задача

Fan-Out/In это технический паттерн, необходимые способы организации потоков данных

Но сами по себе Fan-Out / Fan-In это реализация идеи Worker Pool в Go (решение задачи)

Здесь нужно разобрать вариации использования неиспользования каждого из пары Fan-Out / Fan-In

Случай 1. Нет Fan-Out, Нет Fan-In (Ручное управление). Как выглядит:

Один диспетчер (главная горютина) сам решает, какую задачу какому воркеру отдать, через личный канал каждого воркера. Воркеры не соревнуются за общий канал задач и не собираются в общий канал результатов (пишут ответ в свой собственный канал или слайс).

Round-Robin:

```

type Worker struct {
    id      int
    tasks   chan int
    results chan int // Личный канал для ответа
}

func (w *Worker) Run(wg *sync.WaitGroup) {
    defer wg.Done()
    for task := range w.tasks {
        w.results <- task * 2
    }
}

func main() {
    workers := make([]*Worker, 3)
    for i := 0; i < 3; i++ {
        w := &Worker{id: i, tasks: make(chan int, 1), results: make(chan int, 1)}
        workers[i] = w
        go w.Run(&wg)
    }

    // Диспетчер сам раздает задачи (Fan-Out отсутствует)
}

```

```

    for i := 0; i < 10; i++ {
        workers[i%3].tasks <- i
    }
    // Читаем результаты из личных каналов каждого (Fan-In отсутствует)
    for _, w := range workers {
        close(w.tasks)
    }
    wg.Wait()
    // Собираем результаты вручную из w.results
}

```

Случай 2. Есть Fan-Out, Нет Fan-In (Однонаправленный поток) Как выглядит:

Несколько воркеров читают из общего канала задач (Fan-Out есть), но результаты им не нужно возвращать. Они пишут ответ сразу во внешнюю систему (БД, HTTP-ответ, Kafka) или просто выполняют действие (fire-and-forget).

```

func main() {
    jobs := make(chan string, 100)

    // Fan-Out есть: 5 воркеров дергают письма
    for i := 0; i < 5; i++ {
        go func() {
            for email := range jobs {
                sendEmail(email) // Пишет в SMTP, результат не нужен
            }
        }()
    }

    for _, email := range massiveEmailList {
        jobs <- email
    }
    close(jobs)
    wg.Wait() // Ждем, пока все письма улетят
}

```

Случай 3. Нет Fan-Out, Есть Fan-In (Сток данных)

Как выглядит:

Есть заранее известный набор горутин (например, парсеры сайтов), которые работают параллельно. Fan-Out не нужен, потому что каждая горутина знает свою задачу (свой URL). Но результаты они сливают в один общий канал (Fan-In есть).

```

func main() {
    results := make(chan Page, 10) // Общий сток (Fan-In)
    var wg sync.WaitGroup

    urls := []string{"https://site1.com", "https://site2.com", "https://site3.com"}

    for _, url := range urls {
        wg.Add(1)
        go func(u string) {
            defer wg.Done()
            // Нет Fan-Out: задача жестко закреплена за этой горутинной
            page := fetch(u)
            results <- page // Fan-In: все сливают в один канал
        }(url)
    }
}

```

```
}

// Отдельная горютина для сборки результатов (Fan-In Receiver)
go func() {
    wg.Wait()
    close(results)
}()

for page := range results {
    fmt.Println(page)
}
}
```

Случай 4. Есть Fan-Out и Есть Fan-In (Классика)

разобран в самом начале